

Bluechip User Model Paper

DSN x BCT LLM Agent Challenge - Task A: User Modeling and Review Simulation

Abstract

Bluechip's user model is an evidence-first layer for predicting how a specific user would rate and review an unseen product. The system does not ask an LLM to guess the rating directly. It first builds structured user and item profiles, predicts the rating with deterministic or promoted rating artifacts, plans the review from the predicted rating and evidence, then uses an LLM or fallback template only to express the final review.

The current Task A evidence is conservative and reproducible. The documented all-category rating gate is RMSE 1.2654 on 5,000 examples. A final 25-example all-category generation smoke achieved 1.0 validation consistency, rating mention, item mention, and sentiment alignment under deterministic fallback. The main claim is not that the model is finished, but that the review simulation is measurable, traceable, and grounded before language generation happens.

1. Task Framing

Task A asks for a personalized review and rating simulation agent. This is a user-modeling problem before it is a text-generation problem. A generated review can sound fluent while still implying the wrong score, mentioning unsupported product facts, or ignoring the user's known preferences. Bluechip therefore separates the measurable decision from the prose.

The serving contract is:

```
user evidence + target item evidence
-> user profile
-> item profile
-> rating prediction
-> review plan
-> grounded review text
-> validation and trace
```

This shape makes failure visible. If the output is wrong, we can tell whether the rating model failed, the evidence profile was weak, or the generator drifted from the planned sentiment.

2. User Evidence Model

The user model combines behavioral, textual, and contextual signals:

Signal family: Purpose

Rating behavior: Captures strictness, generosity, and category-specific rating tendency.

Review text: Extracts liked and disliked aspects from prior reviews.

Category affinity: Tracks repeated interest in product categories and domains.

Item evidence: Uses product title, category, metadata, and summary where available.

Nigerian context: Adds locale, budget, delivery, authenticity, and social-proof signals when supported by the persona or history.

Confidence: Exposes when the system has enough user evidence and when it is relying on fallback priors.

The model is intentionally transparent. Instead of hiding all preference learning inside a prompt, Bluechip returns signals and traces that a judge can inspect.

3. Rating-First Review Simulation

The most important design choice is rating-first generation. The system predicts the star rating before writing the review. The review generator then explains the predicted rating using available user and item evidence.

Serving flow:

1. Build a UserProfile from visible history and persona text.
2. Build an ItemProfile from target item metadata and summary.
3. Predict the rating through the promoted Task A serving policy or deterministic fallback.
4. Build a review plan from predicted rating, user signals, item signals, and target item facts.
5. Generate review text through OpenRouter, DeepSeek, OpenAI, or deterministic fallback.
6. Validate rating-review consistency, item grounding, and sentiment alignment.
7. Return the response with confidence, validation result, and trace.

This avoids a common agent failure mode: fluent text that disagrees with the numeric rating. The rating is the measurable decision, and the review is the grounded explanation.

4. Implementation

Important local modules:

Area: Implementation

API: `app/api/routes.py`, `app/main.py`

Task A orchestration: `app/serving/orchestrators/review_simulation.py`

User and item profiling: `app/services/profiling/`, `app/services/intelligence/`

Rating model and features: `app/services/ranking/task_a_model.py`, `app/services/ranking/rating_features.py`

Generation providers: `app/services/generation/`

Validation: `app/services/validation/`

Nigerian context: `app/services/nigerian/`

Traces and metrics: `app/stores/trace_store.py`, `/api/metrics`, `/api/traces`

Evaluation: `eval/eval_task_a.py`, `eval/train_task_a_model.py`, `eval/promote_task_a.py`, `eval/eval_task_a_generation.py`

The repository runs without provider secrets. If no external LLM key is supplied, deterministic fallback generation still produces a valid review and allows judges to test the full API flow.

5. Nigerian Contextualization

The Nigerian layer is evidence-aware rather than decorative. It uses context only when the persona or locale supports it. Signals include price sensitivity in a Naira economy, delivery reliability, authenticity concerns, market versus mall behavior, social proof, and city cues such as Lagos, Abuja, Kano, Ibadan, Port Harcourt, Aba, Onitsha, Enugu, Jos, and Calabar.

For Task A, this affects review voice and evidence selection after rating prediction. A budget-sensitive Lagos student, an Abuja professional buying a gift, and a trader seeking durable inventory should not receive the same explanation style.

6. Evaluation Evidence

Current bounded Task A evidence:

Metric / gate: Value or status
Serving strategy: Rating-first review simulation
Latest documented all-category RMSE gate: 1.2654 on 5,000 examples
Final all-category generation smoke: 25 examples
Validation consistency: 1.0
Rating mention: 1.0
Item grounding: 1.0
Sentiment alignment: 1.0
Fallback behavior: Works without external API keys

The generation smoke is not presented as a full behavioral human evaluation. It is a sanity check that the response remains internally consistent, mentions the target item, and expresses sentiment aligned with the predicted rating.

7. Reproducibility

Core local commands:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
uvicorn app.main:app --reload
```

Task A evaluation commands:

```
python eval/eval_task_a.py
python eval/train_task_a_model.py
python eval/promote_task_a.py
python eval/eval_task_a_generation.py
```

Useful URLs:

<http://127.0.0.1:8000/ui/>

<http://127.0.0.1:8000/docs>

<http://127.0.0.1:8000/api/health>

<http://127.0.0.1:8000/api/metrics>

<http://127.0.0.1:8000/api/traces>

8. Limitations and Next Work

Known limitations:

- The RMSE gate is credible but not state of the art.
- The final generation smoke is small and should not be confused with a large human behavioral study.
- The local artifact registry is production-shaped, but not a managed cloud model registry.
- LLM provider output can improve fluency, but rating quality must remain controlled by evaluated scoring.

Highest-value next work:

1. Expand behavioral human evaluation for review fidelity.
2. Add richer user-level calibration by category and time.
3. Train stronger rating models while preserving fallback reproducibility.
4. Add larger same-slice provider-backed generation evaluation.

9. Rubric Alignment

Brief requirement: User model response

Personalized review simulation: `/api/simulate-review` predicts rating, generates review, validates consistency, and returns trace.

Rating prediction: Rating-first flow with documented RMSE gate.

Grounded generation: Review text is generated from predicted rating and bounded evidence.

Nigerian contextualization: Dedicated context layer uses locale/persona evidence for voice and tradeoffs.

Reproducibility: Runs locally without secrets and exposes deterministic fallback.

Observability: Metrics, traces, validation status, and confidence are returned.

Bluechip's user-model claim is disciplined: build and validate the user's likely decision first, then use language generation to explain that decision clearly.